

Evaluación Unidad 3

Resultado de Aprendizaje:

Depura los algoritmos para ajustar y mejorar la precisión y la eficiencia del análisis de datos.

Indicadores de Logro:

- Identifica errores y limitaciones en algoritmos de análisis de datos, aplicando técnicas de depuración para asegurar su correcto funcionamiento.
- Optimiza estructuras de datos y operaciones algorítmicas para mejorar la eficiencia computacional y reducir tiempos de ejecución.
- Ajusta parámetros y condiciones en los algoritmos, evaluando el impacto de las modificaciones en la precisión de los resultados obtenidos.
- Valida la eficacia de los algoritmos depurados mediante pruebas con diferentes conjuntos de datos, garantizando consistencia, precisión y eficiencia en el análisis.

ENTREGABLES

- **Código fuente** depurado y optimizado (repositorio o carpeta):
 - Módulos .py y/o notebook .ipynb.
 - Scripts de ejecución (run.py o make simple).
- **Informe breve (2–4 páginas)** en PDF:
 - Forma
 - Debe incluir portada, la que debe ser del IP San Sebastián.
 - Interlineado 1.15, letra Arial 12, títulos 14.
 - El documento debe estar con su texto justificado.
 - Trabajo individual
 - Formato Word
 - Bibliografía en formato APA 7
 - Fondo
 - Resumen de cambios (qué y por qué).
 - Exactitud: cómo evitaron duplicación y validaron resultados (tests).
 - Rendimiento: tabla “antes vs. después” con tiempo(s) y memoria (si aplica).
 - Lecciones aprendidas: riesgos, decisiones y límites del enfoque.
- **Carpeta de pruebas** (tests/):
 - 3–5 tests unitarios y 1–2 de regresión (datos sintéticos incluidos).

Evaluación Unidad 3: Depuración y optimización de algoritmos para análisis de datos.

Proyecto: Procesamiento de datos en E-commerce

Contexto:

La empresa **Mercado Austral**, un e-commerce chileno, procesa millones de registros mensuales de pedidos. El pipeline actual calcula métricas como **GMV** (Gross Merchandise Value), **tasa de devoluciones**, **tiempo de cumplimiento** (desde pedido hasta entrega) y **crecimiento por categoría**. El script original “funciona”, pero presenta errores sutiles y bajo rendimiento.

1) Insumos (entregados por el docente)

- **Datasets (CSV)** del mes:
 - orders.csv: order_id, customer_id, order_date, status
 - order_items.csv: order_id, product_id, quantity, unit_price
 - products.csv: product_id, category, subcategory
 - shipments.csv: order_id, shipped_date, delivered_date
 - returns.csv: order_id, return_date (si existe, el pedido fue devuelto)
- **Script base** (módulo Python) con:
 - Cálculo de GMV y tasa de devoluciones con joins que duplican filas (error de exactitud).
 - Cálculo de métricas fila a fila con for/.apply (bajo rendimiento).
 - Parsing de fechas inconsistente (strings mezclados con datetime).
 - Ausencia de manejo de excepciones (el pipeline se cae si falta un CSV).
 - Sin pruebas automatizadas, ni perfilado, ni registro de cambios.

2) Objetivo de la evaluación (lo que deben lograr)

Depurar y optimizar el programa para que:

1. Corrija errores de cálculo (mismas métricas pero correctas).
2. Acelere tiempos de ejecución (evidencia con mediciones antes/después).
3. Mejore mantenibilidad (refactor, DRY, funciones, docstrings, tipos).
4. Agregue robustez (excepciones, logs, tratamiento de datos faltantes).
5. Integre pruebas de regresión/performance que resguarden los resultados.

3) Tareas específicas (paso a paso)

1. Diagnóstico y perfilado

- Ejecutar cProfile/timeit para ubicar cuellos de botella (anexar tablas/figuras simples).

2. Correcciones de exactitud

- Evitar duplicación en métricas al unir orders + order_items + returns (revisar cardinalidades y llaves: order_id único en órdenes, múltiples ítems por orden).
- Asegurar **tasa de devoluciones por orden** (no por ítem).
- Normalizar fechas a datetime y ordenar por order_date.

3. Optimización

- Reemplazar for/.apply por vectorización y groupby/agg.
- Evitar recargas innecesarias de CSV; aplicar *lazy loading* donde aplique.
- Ajustar dtype (enteros, categorías) para memoria.

4. Robustez

- try/except específico (FileNotFoundError, EmptyDataError) con logging.
- Definir comportamiento ante archivos faltantes (p. ej., marcar mes incompleto).

5. Pruebas y validación

- Unitarias: funciones clave (GMV, tasa de devoluciones, lead time).
- Regresión: comparar resultados “antes vs. después” con datos sintéticos.
- (Opcional) Benchmark simple de tiempos.

6. Refactor y estilo

- Separar en funciones/módulos: cargar(), transformar(), calcular_metricas(), reportar().
- Docstrings, tipado opcional, nombres claros, evitar efectos secundarios.